

1 Instructions pour le Projet

- Ce projet est à réaliser **en binôme**. Toutes similitudes suspectes ou plagiat entre les rapports ou codes de groupes différents seront fortement sanctionnées.
- Le langage à utiliser est Prolog (comme en TP). Les commandes vues en cours, TD et TP sont largement suffisantes pour ce projet. Tout appel à des modules ou fonctions non utilisés en TP **est interdit**.
- Votre code doit s'exécuter sans erreur, c'est la base de tout travail en informatique. Un code qui ne s'exécute pas est noté sur la moitié des points.
- Le rendu de votre projet contiendra : un fichier de code `nom1_nom2.pl` et un rapport au format pdf. Le tout sera à mettre dans une archive portant vos deux noms (par exemple .zip), c'est à dire que votre rendu de projet doit se limiter à une seule archive contenant **uniquement** 2 fichiers : un programme et un rapport. Merci de supprimer les exécutables ou autres fichiers.¹
- Trouvez le juste milieu pour le rapport : pas besoin de faire 20 pages, mais 2 pages ne sont pas suffisantes.

Décrivez ce que vous avez fait, vos choix d'implémentations, ce que vous avez compris ou non, ce que font vos clauses, et surtout, *comment utiliser votre programme*. Vous pouvez aussi mettre des captures d'écran de l'exécution de votre programme.

- Le projet est à rendre sur l'espace de dépôt prévu à cet effet pour le **Dimanche 24 Mai à 23h59 au plus tard**, tout retard sera sanctionné².
- Bien entendu, l'usage d'IA (chatGPT etc...) est interdit et sera sanctionné.

1. Tout rendu ne respectant pas cela aura un malus de 2 points.

2. Malus de 2 points par tranche de 12h.

2 Partie 1

Dans ce projet, vous allez manipuler, dans un premier temps, des valeurs binaires : o pour 0 et u pour 1. Les règles de calcul, que vous devrez redéfinir, sont simples et correspondent aux opérations modulo 2 :

- $o + o = o$
- $u + o = u$
- $u + u = o$
- $o \times o = o$
- $u \times o = o$
- $u \times u = u$

Le projet utilisera dans un premier temps des matrices de n lignes et k colonnes, avec des coefficients o et u . Par exemple,

$$\begin{pmatrix} u & o \\ o & u \\ u & u \\ o & u \end{pmatrix}$$

Est une matrice de 4 lignes et 2 colonnes.

Le projet nécessitera de multiplier des vecteurs de taille n (une matrice de 1 ligne et de n colonne) avec des matrices de taille $n \times k$. C'est la multiplication usuelle de matrices (que vous devez connaître). Simplement, les calculs sont faits modulo 2. Ainsi, pour faire le produit $V \times M$:

- partez d'un vecteur ligne de longueur k ne contenant que des o (des 0 donc), appelons ce vecteur R pour "résultat".
- parcourez le vecteur V de gauche à droite, si vous rencontrez un o , ne faites rien. Si vous rencontrez un u en position i , alors ajoutez la i -ème ligne de la matrice (modulo 2) à votre vecteur R .
- À la fin, le vecteur R de longueur k contient donc la somme (modulo 2) de toutes les lignes de la matrice qui sont à des positions où il y a des o dans le vecteur V .

La somme de deux vecteurs contenant des booléens se fait ici terme à terme. Par exemple :

$$(o \ o \ u) + (u \ o \ u) = (u \ o \ o).$$

Pour que les choses soient claires, voici un exemple de produit vecteur-matrice suivant les règles énoncées ci-dessus :

$$(o \ o \ u \ u) \times \begin{pmatrix} u & o \\ o & u \\ u & u \\ o & u \end{pmatrix} = (u \ o).$$

Il est très important que vous repreniez le petit exemple ci-dessus étape par étape pour bien comprendre comment se fait le produit. Il s'agit ici de faire de l'arithmétique modulo 2, tout simplement.

Créez tout d'abord une clause `solution(M,V)` (`M` est une matrice écrite en dur dans votre programme) qui sera vraie quand `V` sera un vecteur tel que $V \times M$ sera égal au vecteur ne contenant que des zéros (`o`).

Par exemple, si `M` est la petite matrice 4×2 donnée en exemple ci-dessus ($M = \begin{pmatrix} u & o \\ o & u \\ u & u \\ o & u \end{pmatrix}$), la requête `solution(M,V)` donnera les réponses suivantes :

`V = (u o u u) ;`
`V = (o u o u) ;`
`V = (u u u o) ;`
`V = (o o o o)`

Dans un second temps, vous modifierez votre programme pour pouvoir spécifier le nombre de “vrais” que vous souhaitez avoir dans votre vecteur `V`.

Vous obtiendrez donc une clause de la forme `solution_binaire(M,V,nombre)` où `nombre` sera le nombre de coefficients valant 1 (c'est à dire `u`) dans `V`.

Par exemple, si `M` est encore la petite matrice 4×2 donnée en exemple ci-dessus, la requête

`solution(M,V,2)`

donnera l'unique réponse suivante :

`V = (o u o u)`

Enfin, vous devrez adapter votre code pour avoir une clause de la forme `solution_binaire(M,V,Y,nombre)`, où Y est un vecteur quelconque tel que $V \times M = Y$ (avec `nombre` étant la contrainte du nombre de u dans le vecteur V encore une fois). **Nous insistons sur le nom de la fonction : toute fonction `solution_binaire(M,V,Y,nombre)` qui aurait un autre nom apporte un malus de 1 point.**

Attention : On vous donne 3 exemples de matrices de taille 26×13 ou 30×15 pour vous entraîner et 3 vecteurs de taille 13 et 15 ; néanmoins votre clause `solution_binaire(M,V,Y,nombre)` doit pouvoir prendre n'importe quelle matrice M et vecteur Y en entrée !

Il est donc important que votre clause analyse les matrices pour s'y adapter, interdiction de faire figurer les valeurs 26 et 13 en dur dans votre code !

N'hésitez pas à faire figurer vos temps dans le rapport, avant ainsi qu'après optimisation, en expliquant bien pourquoi et comment vous avez procédé pour obtenir de meilleurs temps. Si vous avez d'autres idées pour améliorer votre code, décrivez-les, même si vous n'avez pas pu ou su les implémenter.

3 Les matrices et les niveaux de difficulté - Partie 1

Pour rappel, on dit qu'un vecteur V est solution du problème s'il contient le nombre désiré de u dans ses coefficients et si

$$V \times M = (u, o, u, \dots, o) = Y. \quad (\text{par exemple})$$

Les matrices M et vecteurs Y sont données dans le fichier `matrices_binaires.txt`, et voici les objectifs :

M_facile trouvez un vecteur solution V contenant 10 u .

M_moyen trouvez un vecteur solution V contenant 7 u .

M_dur trouvez un vecteur solution V contenant 6 u .

Dans votre rapport, vous devez mettre quelques solutions (c'est à dire des vecteurs V) que vous avez trouvé.

4 Partie 2

Nous allons désormais étendre ce qui a été fait précédemment, dans le cas ternaire. Nous allons donc considérer des valeurs ternaires : o pour 0, u pour 1 et d pour 2. Les règles de calcul correspondent aux opérations modulo 3 :

- $o + o = o$
- $u + o = u$
- $d + o = d$
- $u + u = d$
- $u + d = o$
- $d + d = u$
- $o \times o = o$
- $o \times u = o$
- $o \times d = o$
- $u \times u = u$
- $u \times d = d$
- $d \times d = u$

Pour les opérations entre vecteurs et matrices, ce sont les mêmes principes qu'en binaire, mais les opérations d'addition et de multiplication suivent les règles ternaires.

Le problème est le même, à une différence près. Maintenant, connaissant une matrice M et un vecteur Y , on veut trouver V possédant un certain nombre

nombre de u ou de d . Par exemple, si $M = \begin{pmatrix} u & d \\ d & u \\ u & o \\ o & u \end{pmatrix}$, $Y = (o \ o \ o \ o)$, et que

l'on prend **nombre** égal à 3, alors le vecteur $V = (u \ o \ d \ u)$ convient (2 fois u et 1 fois $d \rightarrow$ on obtient bien 3).

Votre clause s'appellera dans cette partie : `solution_ternaire(M,V,Y,nombre)`.
Nous insistons encore une fois : toute fonction `solution_ternaire(M,V,Y,nombre)` qui aurait un autre nom apporte un malus de 1 point.

5 Les matrices et les niveaux de difficulté - Partie 2

Pour rappel, on dit qu'un vecteur V est solution du problème s'il contient le nombre désiré de u ou de d dans ses coefficients et si

$$V \times M = Y.$$

Les matrices M et vecteurs Y sont données dans le fichier `matrices_ternaires.txt`, et voici les objectifs :

M_facile trouvez des vecteurs solution V contenant 10 u et d .

M_moyen trouvez des vecteurs solution V contenant 8 u et d .

M_dur trouvez des vecteurs solution V contenant 5 u et d .

Encore une fois, mettez des solutions que vous avez trouvé dans le rapport.

6 Indications

Pour commencer, vous devez déclarer les deux valeurs booléennes du projet avec les faits suivants :

```
valeur_binaire(o).  
valeur_binaire(u).
```

Question 1 : Écrivez des faits ou clauses pour obtenir l'addition et la multiplication entre deux valeurs booléennes.

Question 2 : Choisissez une manière pour représenter les vecteurs et matrices de booléens en Prolog, puis écrivez des clauses pour les additions de vecteurs, et multiplication vecteur/matrice.

Question 3 : Écrivez une clause permettant de récupérer tous les vecteurs possibles pour une longueur donnée. Par exemple, la clause

```
vecteur(X,2)
```

donnera les résultats suivants :

```
X = (u,u) ;  
X = (o,u) ;  
X = (u,o) ;  
X = (o,o)
```

Question 4 : À vous de finir le projet. Pour la partie 2, vous devrez suivre la même logique que pour la partie 1, en utilisant un fait `valeur_ternaire` par exemple.